CS 650 – Full Stack Application Develop

8-2 Short Paper: Release Management and Development Strategy

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

August 27, 2026

Release Runbook for a Healthcare Appointment System

Application Context

This runbook governs releases for a full stack healthcare appointment system used by patients, clinicians, and scheduling staff. It supports account access, provider searches, booking, cancellation, reminders, and administrative scheduling. Because failures could delay care, releases must protect availability, data integrity, privacy, and scheduling accuracy.

Versioning Strategy

The team will use Semantic Versioning in the form of MAJOR.MINOR.PATCH. A major version, such as 3.0.0, indicates an incompatible change that may require coordinated client updates, data migration, or revised operating procedures. A minor version, such as 2.4.0, adds backward-compatible functionality, such as a new appointment waitlist. A patch version, such as 2.4.1, contains a backward-compatible correction, such as fixing an incorrect time-zone display. Pre-release identifiers such as 2.5.0-rc.1 will label release candidates that are still being validated and must not be treated as production-ready. One version number will identify the complete releasable package: front end, application programming interface, infrastructure configuration, and compatible database migration set. The same number will appear in the source-control tag, build artifact, deployment record, monitoring dashboard, and release notes. Every release note will identify user-visible changes, known limitations, required migrations, security impact, and rollback compatibility. This consistency allows technical staff and support personnel to determine quickly what is running and whether a change is routine, feature-level, or potentially disruptive.

Repeatable Release Workflow

1. Plan and scope. The product owner defines the release of content and acceptance criteria. Developers link each change to an approved work item, identify dependencies, and classify its risk. Database changes must be used for backward-compatible migrations whenever possible.

2. Build and verify. A protected release branch triggers continuous integration. The pipeline installs locked dependencies, scans for exposed secrets and vulnerable packages, compiles the application, and runs unit, integration, API, user-interface, authorization, and database-migration tests. A failed required check blocks promotion.

3. Validate in staging. The team deploys the immutable release candidate to a production-like staging environment. Quality assurance completes smoke and regression tests for login, patient permissions, provider availability, booking, cancellation, notifications, and audit logging. The security or privacy reviewer verifies that protected health information is not exposed in logs or error messages.

4. Approve and prepare. The release owner confirms test evidence, monitoring readiness, backup status, rollback compatibility, staffing, and the communication plan. A scheduled maintenance window is used for higher risk changes. The team records a go/no-go decision and pauses unrelated production changes.

5. Deploy and observe. The operations lead deploys the approved artifact using the staged strategy below. Automated health checks run after each step. The team monitors errors, latency, authentication failures, appointment transactions, database performance, and notification delivery before increasing traffic.

6. Close and learn. After the observation period, the release owner confirms success, removes the change from freeze, publishes release notes, and records metrics and incidents. Any defect becomes a tracked work item, and a significant failure requires a blameless post-release review. Together, automated checks, staging, independent approval, and production of health gates catch defects before they affect the full user's population.

Deployment Strategy

Canary deployment is appropriate for this system. The current production version remains available while the new version is deployed to a small, isolated set of application instances. The load balancer initially directs approximately 5% of traffic to the canary. Health checks and application monitoring compare the canary with the baseline version. If predefined thresholds remain acceptable, traffic increases in stages, for example to 25%, 50%, and finally 100%. Feature flags keep especially risky user-facing functions disabled until the underlying release proves stable. The main components are immutable versioned artifacts, parallel old and new application instances, a traffic router, automated health checks, centralized logs and metrics, feature flags, and backward-compatible database migrations. This strategy is suitable because appointment scheduling must remain available, and defects can directly affect patient access. A canary limits initial exposure, provides evidence from real production conditions, and permits rapid traffic reversal without waiting for a complete redeployment. Its added infrastructure and monitoring complexity are justified by the system's operational and privacy risks.

Risks, Tradeoffs, and Release Decision

Potential failures include unavailable login or scheduling services, duplicate or missing appointments, incorrect provider availability, delayed reminders, degraded performance, unauthorized data exposure, and a migration that corrupts or locks records. Consequences range

from inconvenience and increased support calls to missed care, privacy harm, compliance exposure, and loss of trust. Dependencies such as identity, email, text-message, or payment services may also fail even when the application build is correct.

The release is safe to proceed only when required tests pass, critical and high-severity vulnerabilities are resolved or formally accepted, backups and restore procedures are verified, migrations are rehearsed, monitoring is active, and the rollback owner is available. The team should define objective stop conditions before deployment. Examples include a statistically meaningful rise in HTTP 5xx errors, failed booking transactions, authentication failures, or latency beyond the service target. Any suspected exposure of protected health information is an immediate no-go or rollback condition. The release owner may accept a minor known defect only when its user impact, workaround, and remediation plan are documented.

Releasing quickly provides faster fixes and feature value and reduces the size of each change when releases are frequent. However, shortening validation or observation increases the likelihood that defects will reach patients. Additional gates improve confidence but add lead time, staffing cost, and process overhead. The team should therefore use risk-based rigor: small reversible patches can move through an expedited path with full automated testing, while identity, permissions, scheduling rules, and schema changes require broader regression testing, longer observation, and explicit approval.

Rollback Plan

The team will detect release problems through automated alerts, canary-versus-baseline dashboards, synthetic booking tests, audit-log review, and reports from clinical or support staff. Alerts must identify the deployed version and notify the release owner and on-call engineer.

When a stop threshold is reached, the release owner halts traffic expansion, declares the rollback in the incident channel, and preserves logs and traces for investigation.

Operations will set the new version's traffic weight to zero and route all requests to the previously verified application instances. The team will disable relevant feature flags, confirm that the prior version passes health and synthetic transaction checks, and validate login, availability search, booking, cancellation, and notifications. If the release included a reversible database migration, the database administrator executes the tested migration only after protecting newly written data and confirming compatibility. Otherwise, the team uses a forward-compatible correction while the prior application version continues operating. Support and clinical stakeholders receive a status update, and normal release activity resumes only after recovery is confirmed. Rollback can still lose incompatible database writes, conflict with cached files, or leave notifications already sent. The team should therefore use expand-and-contract migrations, retain the previous artifact, and test rollback in staging. After recovery, it reconciles appointment records and completes an incident review before another release attempt.